8. Bölüm
Diziler

8.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (data structure) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (array). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (array size) denir. Doğal olarak eleman sayısı tam sayıdır (integer). Elemanların sırasını tam sayı (integer) olan **indis** (index) belirtir. Kümelerin (set) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

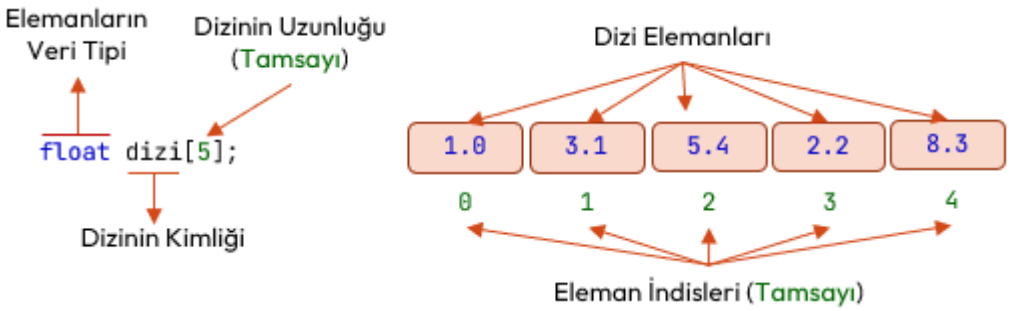
- Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Yani **uzunluğu** (size) tanımlanırken belirlenir.

8.2 Tek boyutlu Diziler

C Dilinde sonlu elemanlı diziler Şekil 8.1'deki gibi tanımlanır;

- **Diziler, benzer veri tipindeki (data type) öğeleri barındırır ve bu öğeler bitişik bellek bölgesini paylaşırlar.**
- Dizi elemanları, ilkel veri tipleri (`char`,`short`, `int`, `long`, `float`, `double`) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı `struct` veya **gösterici** (pointer) olabilir.
- Aynı değer birkaç farklı yerde dizi içinde bulunabilir!
- **Dizinin veri tipi, dizideki elemanlarının veri tipini belirler.** Bir başka deyişle elemanların veri tipi dizinin veri tipiyle aynı olmalıdır.
- **Dizinin uzunluğu (size), tamsayı değişmez (integer literal) olup dizi kimliklendirmesi sırasında belirtilmelidir.** Derleyici buna göre bellekte yer ayırır. Dolayısıyla dizi, bir kez kimliklendirildiğinde dizinin uzunluğu değiştirilemez.
- **Dizi uzunluğu, sabit olmayan bir değişkenle belirlenemez!**
- **Elemanların indisi (index) sıra belirttiğinden her zaman tamsayıdır.**.
- **İlk elemanın indisi `0`, son elemanın indisi dizi uzunluğundan (size) bir küçük olan tamsayıdır.** Bunların dışında bir indis girildiğinde **sınır dışı** (out-of-bounds) hatası yaşanır.



Şekil 8.1: Tek Boyutlu Dizi Tanımlama

C dilinde beş tamsayı elemanı olan bir dizi aşağıdaki şekilde tanımlanır;

```
int tamsayiDizisi[5];
```

Benzer şekilde beş gerçek sayı elemanı olan bir dizi ise aşağıdaki şekilde tanımlanır;

```
float reelsayiDizisi[5];
```

Dizinin elemanlarına tanımlama sırasında **başlangıç değeri** (initial value) süslü parantezler arasında verilebilir yani dizi bazı değerlerle **başlatılabilir** (initialize). Bunun için süslü parantez kullanılır;

```
int tamsayiDizisi1[5]= {1, 2, 3, 4, 5}; //Elemanları 1,2,3,4,5 olacak şekilde başlatıldı.
int tamsayiDizisi2[5]= {0}; //Elemanların hepsi 0 olacak şekilde başlatılıyor.
```

Dizinin elemanlarına hepsine başlangıç değeri verilmeyebilir;

```
float a[5] = {1.0, 2.0, [4] = 12.0}; // 1.Eleman 1.0, 2.Elaman 2.0 ve 5.eleman 12.0
```

Dizinin elemanlarına ayrı ayrı erişilip değerleri değiştirilebilir ya da yeni değer ataması yapılabilir;

```
int a[5]={0}; // a dizisi, tüm elemanları 0 olacak şekilde başlatılıyor
a[0]=1; //Birinci elemana 1 değeri atandı
a[1]++; //İkinci elemanın değeri artırıldı
a[2]--; //Üçüncü elemanın değeri azaltıldı
a[4]=12; //Beşinci elemana 12 değeri atandı.
```

Diziler belleği verimli kullanan ve tek bir değişken ile elemanlara erişim sağlayan bir çözüm sunar. Bir dizideki elemanlar, bellekte bitişik konumda yer aldığından herhangi bir öğeye kolaylıkla erişebiliriz. Dizilerin başlıca üstünlükleri şunlardır:

- İndisleri kullanarak dizi öğelere rastgele ve hızlı erişim sağlanır. Her öğenin bir indisi (index) olduğundan doğrudan erişilebilir ve değiştirilebilir.
- Birden fazla öğeden oluşan tek bir dizi oluşturduğundan daha az kod satırı yazılır.
- Daha az kod satırı yazılarak sıralama yapılabilir.

Dizi kullanmanın zayıf yönleri ise;

- Kimliklendirme sırasında dizi uzunluğuna bir tamsayı değişmez (integer literal) ile karar verildiğinden belli sayıda elemanın üzerinde işlem yapılır.
- Dizi dinamik değildir araya elaman ekleme veya çıkarma yapılamaz.

Örnek olarak klavyeden girilen 5 adet tamsayıyı, giriş sırasının tersinden ekrana yazan C programını verebiliriz;

```
#include<stdio.h>
int main() {
        int dizi[5]; /* 5 elemanı tamsayı olan bir dizi tanımlandı*/
        int indis; /* dizi elemanlarını gezecek ve indis olarak kullanılacak
        tamsayı bir değişken tanımlandı */
        for (indis =0; indis <5; indis ++){
                printf("%d. sayiyi girin:", indis);
                scanf("%d",&dizi[ indis ]); //indis ile belirtilen elemana klavyeden olunan değer
                ↪ atanıyor
        }
        printf("Tersten Sayilar:\n");
        for (indis=4; indis >=0; indis --)
        printf("%d\n", dizi[ indis ]);
        return 0;
}
/*Program Çıktısı:
0. sayiyi girin:10
1. sayiyi girin:20
2. sayiyi girin:40
3. sayiyi girin:50
4. sayiyi girin:35
Tersten Sayilar:
dizi[4]=35
dizi[3]=50
dizi[2]=40
dizi[1]=20
dizi[0]=10
*/
```

Bir başka örnek; Klavyeden girilen 10 adet sınav notuna göre, ortalamanın üstünde olan notları ekrana yazan C programı;

```
#include<stdio.h>
#define BOYUT 10
int main() {
        unsigned notlar[BOYUT]; /* Her terimi pozitif olan 10 elemanlı dizi */
```

```
        float ortalama, toplam=0;
        int indis; //indis için tamsayı değişken
        for (indis=0; indis<BOYUT; indis++){
                printf("%d. notu girin:",indis);
                scanf("%d",&notlar[indis]);
                toplam+=notlar[indis]; // Her eleman girildiğinde toplanıyor
        }
        ortalama=toplam/BOYUT;
        printf("Ortalamanın (%.2f) Üzerindeki Notlar:\n",ortalama);
        for (indis=0; indis<BOYUT; indis++)
        if (notlar[indis]>ortalama)
        printf("%d,",notlar[indis]);
        return 0;
}
/*Program Çıktısı:
0. notu girin:45
1. notu girin:65
2. notu girin:55
3. notu girin:50
4. notu girin:30
5. notu girin:75
6. notu girin:85
7. notu girin:90
8. notu girin:70
9. notu girin:100
Ortalamanın (66.50) Üzerindeki Notlar:
75,85,90,70,100,
*/
```

Bir başka örnek olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani sapma (deviation) ve karesi alınmış sapmalar yani varyans (variance) ile standart sapmayı yazdıran aşağıdaki C programı verilebilir. Ortalamaya olan uzaklıklar yani sapma, verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha yakın gruplaştıklarını gösterir.

```
#include<stdio.h>
#include<math.h>
#define BOYUT 10
int main() {
        float satislar[BOYUT]={100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0};
        float ortalama, toplam=0, varyansToplam=0, standartSapma;
        int indis;
        for (indis=0; indis<BOYUT; indis++)
        toplam+= satislar[indis];
        ortalama=toplam/BOYUT;
        printf("Ortalama:%.2f\n",ortalama);
        printf("Sapma\tVaryans:\n");
        for (indis=0; indis<BOYUT; indis++) {
                float sapma= satislar[indis]-ortalama;
                varyansToplam+=sapma*sapma;
                printf("Sapma: %.2f\tVaryans: %.2f\n", sapma,sapma*sapma);
        }
        standartSapma=sqrt(varyansToplam/BOYUT);
        printf("Standart Sapma:%.2f\n",standartSapma);
        return 0;
}
/*Program Çıktısı:
Ortalama:571.00
Sapma   Varyans:
Sapma: -471.00  Varyans: 221841.00
Sapma: 279.00   Varyans: 77841.00
```

```
Sapma: -71.00   Varyans: 5041.00
Sapma: 29.00    Varyans: 841.00
Sapma: 179.00   Varyans: 32041.00
Sapma: 79.00    Varyans: 6241.00
Sapma: -121.00  Varyans: 14641.00
Sapma: 229.00   Varyans: 52441.00
Sapma: 329.00   Varyans: 108241.00
Sapma: -461.00  Varyans: 212521.00
Standart Sapma:270.50
*/
```

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan tekil (unique) olanlarını bulan program verilebilir.

```
#include <stdio.h>
#define BOYUT 5
int main()
{
        int dizi[BOYUT];
        int i;
        for (i=0; i<BOYUT; i++){
                printf("%d. sayıyı girin:",i);
                scanf("%d",&dizi[i]);
        }
        printf("\nDizi içinde tekil (unique) olan rakamlar: \n");
        for (i=0; i<BOYUT; i++)
        {
                int j, counter = 0;  // Her elemanda sayacı sıfırla
                for (j = 0; j < BOYUT; j++)
                if (i != j) //elemanın kendisini kontrol etmiyoruz
                        if (dizi[i] == dizi[j])
                counter++;
                if (counter == 0)
                printf("%d tekil olarak dizide bulundu.\n", dizi[i]);
        }
        return 0;
}
/*Program Çıktısı:
0. sayıyı girin:12
1. sayıyı girin:15
2. sayıyı girin:12
3. sayıyı girin:3
4. sayıyı girin:15

Dizi içinde tekil (unique) olan rakamlar:
3 tekil olarak dizide bulundu.
*/
```

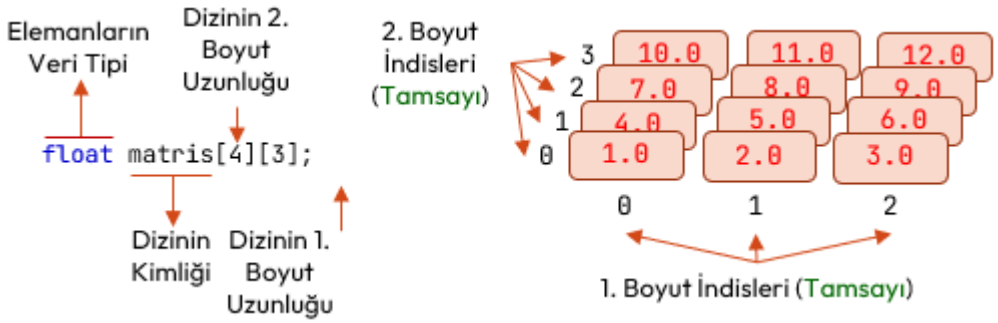
Bir başka örnek olarak 5 elemanlı diziye girilen en büyük elemanına en küçük elemanını ekleyen program verilebilir;

```
#include <stdio.h>
#define BOYUT 5
int main(){
        int dizi[BOYUT]={10,12,3,4,6};
        int enBuyuk, enKucuk, enKucukIndex, enBuyukIndex, i ;
        printf("ÖNCE:"); //Dizinin İlk hali konsola yazdırılıyor.
        for (i = 0; i < BOYUT; i++)
        printf("%02d ",dizi[i]);
        printf("\n");
        enKucuk=dizi[0]; enBuyuk=dizi[0]; /* İlk elemanlar hem en küçük
        hem de en büyük olsun. */
        enKucukIndex=0,enBuyukIndex=0; /* En küçük ve en büyük elemanların
        indisi 0 olsun. */
```

```
        for (i=0; i<BOYUT; i++){ /* En küçük ve en büyük eleman ile
                indislerini bulan kısım: */
                if (dizi[i]<enKucuk) {
                        enKucuk=dizi[i];
                        enKucukIndex=i;
                }
                if (dizi[i]>enBuyuk) {
                        enBuyuk=dizi[i];
                        enBuyukIndex=i;
                }
        }
        printf("En Büyük: dizi[%d]=%02d, En Küçük: dizi[%d]=%02d\n",
        enBuyukIndex,dizi[enBuyukIndex],
        enKucukIndex,dizi[enKucukIndex]);
        dizi[enBuyukIndex]+=dizi[enKucukIndex]; /* En büyük elemana
        en küçüğünü ekleme: */
        printf("SONRA:"); //Dizinin son hali konsola yazdırılıyor.
        for (i = 0; i < BOYUT; i++)
        printf("%02d ",dizi[i]);
        return 0;
}
/*Program Çıktısı:
ÖNCE:10 12 03 04 06
En Büyük: dizi[1]=12, En Küçük: dizi[2]=03
SONRA:10 15 03 04 06
*/
```

8.3 İki Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (two-dimentional array) matematikten de bildiğimiz üzere matris (matrix) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.



Şekil 8.2: İki Boyutlu Dizi Tanımlama

Örneği verilen iki boyutlu dizinin ikinci boyutundan bakacak olursak her biri ayrı bir tek boyutlu dizidir. Bellekte ikici boyutu oluşturan bu diziler de sırasıyla birbirine bitişik (contigious) olarak yer alır. Aslında `float dizi[3][4];` ile `float dizi[12];` aynı bellek yerleşimine sahiptir. Aşağıda iki boyutlu matris tanımlamalarına örnek verilmiştir;

```
float matris[2][3];  //Her bir elemanı 3 elemanlı bir dizi olan 2 satır üç sütunlu bir matris
//Aşağıda 2x3 matrisi başlatma:
float matris2[2][3]= {
        {1.0,2.0,3.0}, //Birinci Satır: 3 Elemanlı bir dizi ile başlatılıyor
        {2.0,4.0,6.0}  //İkinci Satır: 3 Elemanlı bir başka dizi ile başlatılıyor.
};
int karematris[2][2];
//2x2 matrise ilk değer verme:
int karematris2[2][2]= {
        {1,2}, //Birinci Satır: 2 elemanlı bir dizi
        {5,6} //Birinci Satır: 2 elemanlı bir dizi
};
```

```
int karematris3[3][3];
```

Örnek olarak 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
#include<stdio.h>
#define SATIR 3
#define SUTUN 4
int main() {
        int matris[SATIR][SUTUN];
        int i,j;
        /*
        //İç içe döngü kullanmasaydık:
        for (j=0; j<SUTUN; j++)  //Birinci Satır Okuyalım
                scanf("%d",&matris[0][j]);
        for (j=0; j<SUTUN; j++)  //İkinci Satır Okuyalım
                scanf("%d",&matris[1][j]);
        for (j=0; j<SUTUN; j++)  //Üçüncü Satır Okuyalım
                scanf("%d",&matris[2][j]);
        //Bunun yerine aşağıdaki gibi iç içe iki for tanımlanır;
        */
        for (i=0; i<SATIR; i++)  //İkinci Boyut İçin
                for (j=0; j<SUTUN; j++)  //Birinci Boyut İçin
                        scanf("%d",&matris[i][j]);
        printf("TABLO\n");
        for (i=0; i<SATIR; i++) {  //İkinci Boyut İçin
                for (j=0; j<SUTUN; j++)  //Birinci Boyut İçin
                        printf("%d\t",matris[i][j]);
                printf("\n");
        }
        return 0;
}
/*Program Çıktısı:
3 4 5 6 7 8 4 4 5 8 7 9
TABLO
3       4       5       6
7       8       4       4
5       8       7       9
*/
```

Bir başka örnek olarak elemanları klavyeden girilen 3x2'lık matrisin satır ve sütun toplamlarını ekrana yazan programı verilebilir;

```
#include<stdio.h>
#define SATIR 3
#define SUTUN 2
int main() {
        int matris[SATIR][SUTUN]; //Matris Tanımı
        int i,j;  //i sutun için, j satır için tanımlandı
        for (i=0; i<SATIR; i++) {  //İkinci Boyut (SATIR) İçin
                printf("%d. satir:",i);
                for (j=0; j<SUTUN; j++)  //Birinci Boyut (SUTUN) İçin
                        scanf("%d",&matris[i][j]);
        }
        int satirToplamlar[SATIR]={0}; //Bütün elemanları 0 olan dizi
        for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
                for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
                        satirToplamlar[i]+=matris[i][j];
                        /* İçdeki for bitince tüm satırdaki elemanlar toplanmış oldu */
        }/* Dışdaki for ile de her bir satır için toplam tekrarlanıyor */
        printf("\nSatır Toplamları:");
        for (i=0; i<SATIR; i++)
        printf("%d\t",satirToplamlar[i]);
        int sutunToplamlar[SUTUN]={0}; //Bütün elemanları 0 olan dizi
```

```
        for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
                for (i=0; i<SATIR; i++) {  // İkinci Boyut İçin
                        sutunToplamlar[j]+=matris[i][j];
                        /* İçdeki for bitince tüm sutundaki elemanlar toplanmış oldu*/
                }/* Dışdaki for ile de her bir sutun için toplam tekrarlanıyor */
        printf("\nSütun Toplamları:");
        for (j=0; j<SUTUN; j++)
                printf("%d\t",sutunToplamlar[j]);
        return 0;
}
/*Program Çıktısı:
0. satir:12 18
1. satir:22 28
2. satir:2 8

Satır Toplamları:30     50      10
Sütun Toplamları:36     54
*/
```

8.4 Çok Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu diziler veya iki boyutlu diziler olan matrislerdir. Çok boyutlu dizilere örnek olarak En-Boy-Yükseklik içeren 3 boyutlu diziler verilebilir. Üç boyutlu dizilerde dizinin her bir elemanı bir matristir. Çok boyutlu dizilerde de en içteki dizinin boyutu kimliklendirmede sağda yer alır. Aşağıdaki örnekte; çeşitli boyutlarda diziler tanımlanmıştır;

```
int ucbuyutludizi1[3][2][2]; // ucbuyutludizi1: elemanları 3 adet 2x2 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[2][3][3]; // ucbuyutludizi2: elemanları 2 adet 3x3 matris olan üç boyutlu bir
↪ dizi
int ucbuyutludizi1[4][3][2]; // ucbuyutludizi3: elemanları 4 adet 3x2 matris olan üç boyutlu bir
↪ dizi
int dortbuyutludizi1[5][2][3][2]; /* dortbuyutludizi1: elemanları 5 adet olan ve her bir elemanı
↪ 2 adet 3x2 matris olan dört boyutlu bir dizi */
```

Aşağıda üç boyutlu dizilerde bir ilk değer verme kod örneği verilmiştir;

```
#define DERINLIK 3
#define SATIR 3
#define SUTUN 3
int ucBoyutluDizi[DERINLIK][SATIR][SUTUN]={
        { {1,2,3},{4,5,6},{7,8,9} },
        { {11,12,13},{14,15,16},{17,18,19} },
        { {21,22,23},{24,25,26},{27,28,29} }
};
```

Çok boyutlu dizi örneği olarak şöyle bir örnek verilebilir; 3 farklı ders alan 10 öğrenci, her bir dersten 4 değişik not almaktadır. Bu notların ağırlıkları sırasıyla %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan C programı aşağıdaki gibidir;

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define KACDERS 3
#define KACOGRENCI 10
#define KACDEGISIKNOT 4
int main() {
        int notlar[KACDERS][KACDEGISIKNOT][KACOGRENCI];
        int agirlik[KACDEGISIKNOT]={10,30,20,40};
        //Ödevler:%10, Vize:%30, Hızlı Sınav:%20, Final:%40
        int i,j,k; //indis değişkenleri
        srand(time(NULL)); // rastgele için
        for (k=0; k< KACDERS; k++)
        for (i=0; i< KACOGRENCI; i++)
```

```
        for (j=0; j< KACDEGISIKNOT; j++)
        notlar[k][j][i]=rand()%101;
        //Her bir sınavın Ortalaması hesaplanacak:
        for (k=0; k< KACDERS; k++) {
                printf("%d.Ders İçin:\n",k);
                for (j=0; j<KACDEGISIKNOT; j++)  {
                        float sinavToplam=0.0;
                        for (i=0; i<KACOGRENCI; i++) {
                                sinavToplam+=notlar[k][j][i];
                        }
                        printf("%d. Sınav ortalaması:%.2f\n",j,sinavToplam/KACOGRENCI);
                }
        }
        //Her bir Öğrencinin Ağırlıklı Notu hesaplanacak:
        for (k=0; k< KACDERS; k++) {
                printf("%d.Ders İçin:\n",k);
                for (i=0; i<KACOGRENCI; i++) {
                        float agirlikliNot=0.0;
                        for (j=0; j<KACDEGISIKNOT; j++)  {
                                agirlikliNot+=notlar[k][j][i]*agirlik[j]/100.0;
                        }
                        printf("%d. Öğrenci ortalaması:%.2f\n",i,agirlikliNot);
                }
        }
        return 0;
}
/*Program Çıktısı:
0.Ders İçin:
0. Sınav ortalaması:48.40
1. Sınav ortalaması:51.40
2. Sınav ortalaması:57.10
3. Sınav ortalaması:54.60
1.Ders İçin:
0. Sınav ortalaması:58.60
1. Sınav ortalaması:40.30
2. Sınav ortalaması:74.00
3. Sınav ortalaması:46.60
2.Ders İçin:
0. Sınav ortalaması:59.40
1. Sınav ortalaması:54.20
2. Sınav ortalaması:49.20
3. Sınav ortalaması:36.00
0.Ders İçin:
0. Öğrenci ortalaması:52.90
1. Öğrenci ortalaması:30.10
2. Öğrenci ortalaması:66.40
3. Öğrenci ortalaması:79.70
4. Öğrenci ortalaması:57.80
5. Öğrenci ortalaması:46.10
6. Öğrenci ortalaması:27.70
7. Öğrenci ortalaması:70.20
8. Öğrenci ortalaması:53.10
9. Öğrenci ortalaması:51.20
1.Ders İçin:
0. Öğrenci ortalaması:33.50
1. Öğrenci ortalaması:37.30
2. Öğrenci ortalaması:74.90
3. Öğrenci ortalaması:57.30
4. Öğrenci ortalaması:46.40
5. Öğrenci ortalaması:48.10
6. Öğrenci ortalaması:32.40
7. Öğrenci ortalaması:71.90
```

```
8. Öğrenci ortalaması:59.00
9. Öğrenci ortalaması:53.10
2.Ders İçin:
0. Öğrenci ortalaması:47.10
1. Öğrenci ortalaması:39.90
2. Öğrenci ortalaması:50.50
3. Öğrenci ortalaması:40.70
4. Öğrenci ortalaması:67.10
5. Öğrenci ortalaması:14.90
6. Öğrenci ortalaması:36.00
7. Öğrenci ortalaması:36.90
8. Öğrenci ortalaması:45.10
9. Öğrenci ortalaması:86.20
*/
```

8.5 Parametre Olarak Diziler

Parametre olarak gönderilen dizinin boyutu kadar köşeli parantez açılır ve kapatılır, ilk köşeli parantez içerisine eleman sayısını ifade eden değer yazılmaz, fakat diğerlerine eleman sayıları verilmek zorundadır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir. Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```
void diziIsleyenFonksiyon1(int tekBoyutluDizi[],int boyut);
void diziIsleyenFonksiyon2(int pDizi[][2],int ikinciBoyut,int birinciBoyut);
void diziIsleyenFonksiyon3(int pDizi[][3][2],int ucuncuBoyut,int ikinciBoyut,int birinciBoyut);

// Bu fonksiyonları aşağıdaki şekilde çağırabiliriz;
unsigned notlar[10]; //10 öğrenci notu barındırır.
unsigned dersNotlari[2][10]; //2 dersi alan 10 öğrenci için notlar;
unsigned bolumDersNotlari[4][2][10]; /* 4 bölümde 2 dersi alan 10 öğrenci için notlar;*/
//...
diziIsleyenFonksiyon1(notlar,10);
diziIsleyenFonksiyon2(dersNotlari,2,10);
diziIsleyenFonksiyon3(bolumDersNotlari,4,2,10);
//...
```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```
#include<stdio.h>
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);
int main() {
        int dizi[5];
        float ortalama;
        diziOku(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
        diziYaz(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
        ortalama=diziOrtalama(dizi,5); /* fonksiyon geri dönüş değeri bir değişkene atanıyor.
         ↪  Atandığı değişkenin tipi ile fonksiyonun geri dönüş tipi aynı olmalı! */
        printf("Dizi Ortalaması: %.2f",ortalama);
        return 0;
}
void diziOku(int pDizi[],int pUzunluk){
        int sayac;
        printf("%d Elemanlı Dizi Okunacaktır:",pUzunluk);
        for (sayac=0; sayac < pUzunluk; sayac++)
                scanf("%d",&pDizi[sayac]);
}
void diziYaz(int pDizi[],int pUzunluk){
        int sayac;
        printf("%d Elemanlı Dizi:\n",pUzunluk);
        for (sayac=0; sayac < pUzunluk; sayac++)
                printf("%4d",pDizi[sayac]);
```

```
        printf("\n");
}
float diziOrtalama(int pDizi[],int pUzunluk){
        int sayac;
        float toplam=0;
        for (sayac=0; sayac < pUzunluk; sayac++)
                toplam+=pDizi[sayac];
        return toplam/pUzunluk;
}
/*Program Çıktısı:
5 Elemanlı Dizi Okunacaktır:10 20 30 40 50
5 Elemanlı Dizi:
10  20  30  40  50
Dizi Ortalaması: 30.00
*/
```

8.6 Dizi Elemanlarını Sıralama

Dizilerle ilgili bazı durumlarda elemanları sıralamak isteriz. Birden çok sıralama yöntemi vardır; Kabarcık sıralama (bubble sort), Seçme sıralama (selection sort), Araya sokma sıralama (insertion sort), Shell sıralaması (Shell sort) ve Çabuk sıralama (quick sort) gibi.

§8.6.1 Kabarcık Sıralama

Kabarcık sıralama (bubble sort) algoritmasında;

1. Her bir eleman kendisinden sonra gelen eleman ile karşılaştırılır.
2. Büyük ya da küçük sorgulaması yapılır. Büyük sorgulaması yapıldığında dizi küçükten büyüğe, küçük sorgulaması yapıldığında dizi büyükten küçüğe sıralanır.
3. Eğer şarta uyuyor ise elemanların yerleri değiştirilir (swap).
4. Bir sonraki elemana geçilerek birinci adımdan bu adıma kadar olan adımlar son dizi elemanına kadar devam edilir.
5. Yukarıdaki dört adım bitirildiğinde yalnızca birinci eleman doğru yerde olur. Böylece algoritmanın birinci aşaması tamamlanmıştır. Dolayısıyla bir sonraki elemana geçilerek dizinin eleman sayısı kadar bu işlemin tekrar edilmesi gerekir. Son eleman için aşamanın tekrar edilmesine gerek yoktur. Dolayısıyla aşama sayısı dizi uzunluğundan bir eksik yapılır.

Bunu bir örnekle açıklayalım;

```
int dizi[5]={14,11,0,3,7}; //Dizisini Küçükten Büyüğe sıralayalım:

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 14>11 ise yer değiştir
// Dizinin yeni hali: {11,14,0,3,7}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 14>0 ise yer değiştir
// Dizinin yeni hali: {11,0,14,3,7}
if (a[2]<a[3]) { int gecici=a[2]; a[2]=a[3]; a[3]=gecici; } // 14>3 ise yer değiştir
// Dizinin yeni hali: {11,0,3,14,7}
if (a[3]<a[4]) { int gecici=a[3]; a[3]=a[4]; a[4]=gecici; } // 14>7 ise yer değiştir
// Dizinin yeni hali: {11,0,3,7,14}
//Buraya kadar birinci aşama tamamlanmıştır. En son eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 11>0 ise yer değiştir
// Dizinin yeni hali: {0,11,3,7,14}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 11>3 ise yer değiştir
// Dizinin yeni hali: {0,3,11,7,14}
if (a[2]<a[3]) { int gecici=a[2]; a[2]=a[3]; a[3]=gecici; } // 11>7 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar ikinci aşama tamamlanmıştır. Sondan ikinci eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 0>3 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 3>7 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
```

```
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar üçüncü aşama tamamlanmıştır. Sondan üçüncü eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 0>3 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar dördüncü aşama tamamlanmıştır. Sondan dördüncü eleman belirlenmiştir.

// Bir aşamaya daha gerek yoktur. Çünkü kalan 1 elemanın yeri değişmeyecektir.
```

Şimdi de sıralamayı döngü ile yapalım;

```
#include <stdio.h>
#define UZUNLUK 5
int main() {
        int dizi[UZUNLUK]={14,11,0,3,7};
        int indis;
        printf("Dizinin SıralanMAmış hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        printf("\n");
        int asama;
        for (asama=0; asama<UZUNLUK-1; asama++) { //asama için
                for (indis=0; indis<UZUNLUK-asama-1;indis++) { /* karşılaştırma
                        asama kadar eksik
                        yapılır */
                        if (dizi[indis] > dizi[indis+1]) { /* birinci eleman ile
                                ikincisini karşılaştır */
                                int yedek=dizi[indis]; // birinci elemanı yedekle
                                dizi[indis]=dizi[indis+1]; /* birinci elemana
                                ikicici elemanı ata */
                                dizi[indis+1]=yedek;  // yedeği ikinci elemana ata
                        }
                }
        }
        printf("Dizinin SıralanMIŞ hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        return 0;
}
/*Programın Çıktısı:
Dizinin SıralanMAmış hali:
14, 11, 0, 3, 7,
Dizinin SıralanMIŞ hali:
0, 3, 7, 11, 14,
*/
```

Bu sıralama türü küçük veri setleri veya eğitim amaçlı olarak kullanılır.

§8.6.2 Seçme Sıralama

Seçme sıralama (selection sort) algoritmasında;

1. Küçükten büyüğe seçme sıralama aşağıdaki adımlar izlenerek yapılır;
   (a) Dizi üzerinde ilk elemandan başlanarak dizi sonuna kadar ilerlenir.
   (b) İlerleme sırasındaki elemanın sonrasındaki elemanlar arasında en küçük eleman bulunur.
   (c) Bulunan en küçük eleman ile ilerleme sırasındaki eleman yer değiştirilir.
2. Büyükten küçüğe seçme sıralama aşağıdaki adımlar izlenerek yapılır;
   (a) Dizi üzerinde ilk elemandan başlanarak dizi sonuna kadar ilerlenir.
   (b) İlerlemeye sırasındaki elemanın sonrasındaki elemanlar arasında en büyük eleman bulunur.
   (c) Bulunan en küçük eleman ile ilerleme sırasındaki eleman yer değiştirilir.

Aşağıda buna ilişkin örnek program verilmiştir;

```
#include <stdio.h>
#define UZUNLUK 5
void diziYaz(int pDizi[], int pBoyut);
```

```
void selectionSortKucuktenBuyuge(int pDizi[], int pBoyut);
void selectionSortBuyuktenKucuge(int pDizi[], int pBoyut);
int main() {
        int dizi[UZUNLUK]={14,11,0,3,7};
        printf("Dizinin SıralanMAMIŞ hali:\n");
        diziYaz(dizi, UZUNLUK);
        selectionSortKucuktenBuyuge(dizi,UZUNLUK);
        printf("Dizinin Küçükten Büyüğe SıralanMIŞ hali:\n");
        diziYaz(dizi, UZUNLUK);
        selectionSortBuyuktenKucuge(dizi,UZUNLUK);
        printf("Dizinin Büyüktem Küçüğe SıralanMIŞ hali:\n");
        diziYaz(dizi, UZUNLUK);
        return 0;
}
void diziYaz(int pDizi[], int pBoyut){
        int sayac;
        for (sayac=0; sayac<pBoyut; sayac++)
        printf("%d,",pDizi[sayac]);
        printf("\n");
}
void selectionSortKucuktenBuyuge(int pDizi[], int pBoyut){
        int ilerleme;
        for (ilerleme=0; ilerleme<pBoyut-1; ilerleme++) {
                int sayac;
                int enKucukIndis=ilerleme;
                for (sayac=ilerleme+1; sayac<pBoyut; sayac++)/* Geri kalanındaki
                en Küçüğün İndisi */
                if (pDizi[sayac] < pDizi[enKucukIndis])
                enKucukIndis =sayac;
                /* Bulunan en küçükindisli eleman ile
                ilerlemedeki eleman yer değiştirilir */
                int yedek=pDizi[ilerleme]; /* yer değiştirme:
                yedek, dizi veri tipinde olmalı */
                pDizi[ilerleme]=pDizi[enKucukIndis];
                pDizi[enKucukIndis]=yedek;
        }
}
void selectionSortBuyuktenKucuge(int pDizi[], int pBoyut){
        int ilerleme;
        for (ilerleme=0; ilerleme<pBoyut-1; ilerleme++) {
                int sayac;
                int enBuyukIndis=ilerleme;
                for (sayac=ilerleme+1; sayac<pBoyut; sayac++)  /*Geri kalanındaki
                en Küçüğün İndisi */
                if (pDizi[sayac] > pDizi[enBuyukIndis])
                enBuyukIndis =sayac;
                /* Bulunan en küçükindisli eleman ile
                ilerlemedeki eleman yer değiştirilir */
                int yedek=pDizi[ilerleme]; //yer değiştirme:
                pDizi[ilerleme]=pDizi[enBuyukIndis];
                pDizi[enBuyukIndis]=yedek;
        }
}
/*Program Çıktısı:
Dizinin SıralanMAMIŞ hali:
14,11,0,3,7,
Dizinin Küçükten Büyüğe SıralanMIŞ hali:
0,3,7,11,14,
Dizinin Büyüktem Küçüğe SıralanMIŞ hali:
14,11,7,3,0,
*/
```

Bu sıralama türü, yer değiştirme sayısı az olduğu için bellek yazma işleminin maliyetli olduğu durumlarda

kullanılır.

§8.6.3 Araya Sokma Sıralaması

Araya sokma sıralamasında (insertion sort), diziyi hayali olarak **sıralı** ve **sıralanmamış** olarak ikiye böler. Sıralanmamış kısımdan her seferinde bir eleman alır ve sıralı kısımdaki doğru yerine yerleştirir.

```
#include <stdio.h>
#define UZUNLUK 5
int main() {
        int dizi[UZUNLUK]={14,11,0,3,7};
        int indis;
        printf("Dizinin SıralanMAmış hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        printf("\n");

        //insertionSort
        int i, key, j;
        for (i = 1; i < UZUNLUK; i++) {
                key = dizi[i]; // Sıralanacak eleman
                j = i - 1;
                // Key'den büyük olan elemanları bir sağa kaydır
                while (j >= 0 && dizi[j] > key) {
                        dizi[j + 1] = dizi[j];
                        j = j - 1;
                }
                dizi[j + 1] = key; // Boşalan yere key'i yerleştir
        }

        printf("\nDizinin SıralanMIŞ hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        return 0;
}
```

Bu sıralama türü, veriler **neredeyse sıralıysa** çok hızlı çalışır.

§8.6.4 Shell Sıralaması

Shell sıralaması (Shell sort), aslında araya sokma algoritmasının çok daha zekice ve hızlandırılmış bir versiyonudur. 1959 yılında *Donald Shell* tarafından geliştirilmiştir.

Araya sokma sıralamasının en büyük zayıflığı, elemanları sadece bir yanındakiyle kıyaslamasıdır. Eğer dizinin en sonundaki küçük bir elemanı en başa taşımak isterseniz, algoritma onu tek tek tüm elemanlarla kaydırarak götürür. Shell sıralaması bu sorunu şöyle çözer:

- Diziyi belirli bir aralık (gap) değeriyle parçalara böler (örneğin dizinin yarısı kadar).
- Birbirinden uzak olan bu elemanları kendi aralarında sıralar.
- Aralığı kademeli olarak azaltır (genelde her adımda yarıya düşürür).
- Aralık 1 olduğunda, dizi zaten **neredeyse sıralı** hale gelmiş olur. Son adımda standart bir araya sokma sıralaması yapar ve işlem biter.

```
#include <stdio.h>
void shellSort(int dizi[], int n) {
        // Aralığı (gap) dizinin yarısından başlat, her seferinde yarıya düşür
        for (int gap = n / 2; gap > 0; gap /= 2) {
                // Bu aralık değeri için araya sokma sıralaması yap
                for (int i = gap; i < n; i++) {
                        int temp = dizi[i];
                        int j;
                        // Elemanları gap mesafesiyle kıyasla ve kaydır
                        for (j = i; j >= gap && dizi[j - gap] > temp; j -= gap) {
                                dizi[j] = dizi[j - gap];
                        }
                        // temp (başta saklanan eleman) doğru yere yerleştirilir
                        dizi[j] = temp;
```

```
                }
        }
}
int main() {
        int dizi[] = {23, 12, 1, 8, 34, 54, 2, 3};
        int n = sizeof(dizi) / sizeof(dizi[0]);

        shellSort(dizi, n);

        printf("Shell Sort ile siralanmis dizi: ");
        for (int i = 0; i < n; i++)
                printf("%d ", dizi[i]);

        return 0;
}
```

§8.6.5 Çabuk Sıralama

Çabuk sıralama (quick sort), bir pivot eleman seçer. Pivotun soluna ondan küçükleri, sağına ise ondan büyükleri toplar. Ardından aynı işlemi oluşan iki küçük grup için özyinelemeli (recursive) olarak tekrarlar.

```
#include <stdio.h>
// Bu örnek için Göstericiler bilinmelidir!
void takas(int* a, int* b) {// İki elemanın yerini değiştiren yardımcı fonksiyon
        int t = *a;
        *a = *b;
        *b = t;
}
int partition(int dizi[], int low, int high) { // Diziyi parçalara ayıran fonksiyon
        int pivot = dizi[high]; // Son elemanı pivot seçtik
        int i = (low - 1); // Küçük elemanların indisi

        for (int j = low; j <= high - 1; j++) {
                if (dizi[j] < pivot) {
                        i++;
                        takas(&dizi[i], &dizi[j]);
                }
        }
        takas(&dizi[i + 1], &dizi[high]);
        return (i + 1);
}
void quickSort(int dizi[], int low, int high) {
        if (low < high) {
                int pi = partition(dizi, low, high); // Pivotun doğru yerini bul
                quickSort(dizi, low, pi - 1);  // Pivotun solunu sırala
                quickSort(dizi, pi + 1, high); // Pivotun sağını sırala
        }
}
#define UZUNLUK 5
int main() {
        int dizi[UZUNLUK]={14,11,0,3,7};
        int indis;
        printf("Dizinin SıralanMAmış hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        printf("\n");

        quickSort(dizi,0, UZUNLUK-1);

        printf("\nDizinin SıralanMIŞ hali:\n");
        for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
        return 0;
```

```
}
```

Bu sıralama türü genel amaçlı en hızlı algoritmalardan biridir. Binlerce veya milyonlarca veriyi sıralayacaksan vazgeçilmezdir. Ortalama çalışma süresi $O(n \log n)$ olduğu için devasa fark yaratır.

§8.6.6 Hazır Sıralama Fonksiyonları

C dilinde bu algoritmaları elle yazmak yerine, standart kütüphanede (`stdlib.h`) bulunan çok hızlı ve optimize edilmiş `qsort()` fonksiyonunu kullanabilirsiniz;

```c
#include <stdio.h>
#include <stdlib.h> //qsort() için

int karsilastir(const void *a, const void *b) { // Karşılaştırma fonksiyonu (qsort için
↪ gereklidir)
        return (*(int*)a - *(int*)b);
}
int main() {
        int dizi[] = {42, 15, 8, 99, 23};
        int n = sizeof(dizi) / sizeof(dizi[0]);
        // qsort(dizi_adresi, eleman_sayisi, eleman_boyutu, karsilastirma_fonksiyonu)
        qsort(dizi, n, sizeof(int), karsilastir);
        printf("Sıralı dizi: ");
        for(int i = 0; i < n; i++) printf("%d ", dizi[i]);
        return 0;
}
```

8.7 Düşün ve Çöz

- **Düşün**: Çok boyutlu dizilerin (matrislerin) bellekte aslında tek boyutlu ve ardışık olarak tutulmasının, dizi elemanlarına erişim hızına etkisini tartışınız.
- **Düşün**: C dilinde dizi sınırlarının (out of bounds) kontrol edilmemesinin, bellek güvenliği ve "buffer overflow" saldırıları açısından riskini tartışınız.
- **Düşün**: İkili Arama (binary search) algoritmasının çalışabilmesi için dizinin neden mutlaka sıralı olması gerektiğini açıklayınız.
- **Çöz**: 10 elemanlı rastgele sayılardan oluşan bir diziyi kabarcık sıralama algoritması ile küçükten büyüğe sıralayan fonksiyonu yazınız.
- **Çöz**: 20 elemanlı bir dizideki elemanların frekansını (hangi sayıdan kaç tane olduğunu) bulan ve ekrana listeleyen programı yazınız.
- **Çöz**: Kullanıcıdan alınan iki adet 3x3'lük matrisin toplamını bulan ve sonucu ekrana matris formunda basan fonksiyonu kodlayınız.